



UNIVERSITY OF
TORONTO

Applied Deep Learning

Lecture 1: *Fundamentals of DL*

Ali Bereyhi

ECE Department, University of Toronto

Fall 2026



UNIVERSITY OF
TORONTO

Lecture Overview



Lecture 1: *Preliminaries and Fundamentals of DL*

Today's lecture covers

1. Basics of machine learning and deep learning
2. Components of machine learning
3. Model training via risk minimization
4. Universal approximation by neural networks



UNIVERSITY OF
TORONTO

Preliminaries

Machine Learning (ML)

Goodfellow et al. *informally* define ML as “. . . a form of applied statistics with increased emphasis on the use of computers to statistically estimate complicated functions. . .”

Though good, this definition is still unclear! Let's put it in simple words

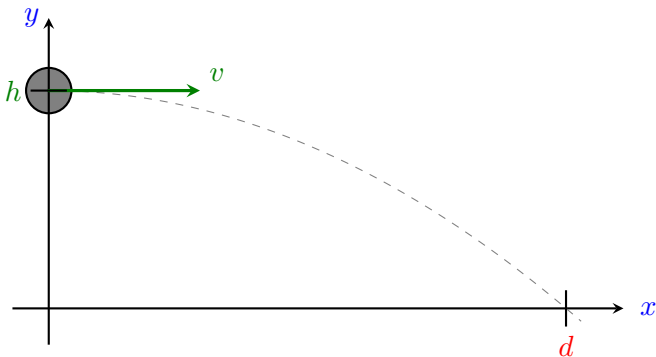
For most problems, there are two approaches to get to a solution

- through **analytic** derivation
- using **data-driven** algorithms

ML develops efficient **data-driven** algorithms for *complicated problems* whose analysis is either *infeasible* or *too complicated*

Motion of a Projectile

A projectile, located at height h , is initiated by horizontal velocity v . We are asked to determine horizontal distance d at which it hits the ground



Motion of a Projectile: *Analytic Solution*

At time t , the location of the projectile (x, y) is

$$x = vt \qquad y = -\frac{g}{2}t^2 + h$$

with $g \approx 9.8 \text{ m/s}^2$ being the gravitational acceleration. When it hits the ground, we have $y = 0$; thus, we can find the hitting time as

$$t_0 = \sqrt{\frac{2h}{g}} = \gamma\sqrt{h}$$

At $t = t_0$, we have $x = d = vt_0$. Defining $\gamma = \sqrt{2/g} \approx 0.45$, we have

$$d = \gamma v \sqrt{h}$$

Motion of a Projectile: *Analytic Solution*

The projectile hits the ground at the horizontal distance

$$d = \gamma v \sqrt{h}$$

To derive this analytic result, we have used two facts

- We knew Newton's laws that describe the motion
- We could solve the equations for d analytically

Well! This is not the case in all problems! In fact, there might be

- **no analytic law known** that describes the relations, or
- a **complicated law** whose analysis is computationally **infeasible**

Motion of a Projectile: *ML Approach*

We conduct the experiment for I different times: in try $i = 1, \dots, I$

- we initiate with different velocity v_i and height h_i
- we measure the horizontal distance d_i

We then assume that d_i and $[v_i, h_i]$ are related via a *pre-defined function*; for instance, we assume d_i and $[v_i, h_i]$ are related as

$$d_i = w_0 v_i + w_1 h_i$$

We then try to find the optimal values

$$w_0 = w_0^* \text{ and } w_1 = w_1^*$$

such that *pre-defined function closely matches our experimental results*

Motion of a Projectile: *ML Approach*

How can we find $w_0^\star$ and $w_1^\star$?

The function should match our experimental data, i.e., *for any i*

$$d_i \stackrel{!}{=} w_0^\star v_i + w_1^\star h_i \rightsquigarrow (d_i - w_0^\star v_i - w_1^\star h_i)^2 \stackrel{!}{=} 0$$

By $\stackrel{!}{=}$, we mean that we intend to have this identity holding

Having this identity *for any i* is equivalent to write

$$\sum_{i=1}^I (d_i - w_0^\star v_i - w_1^\star h_i)^2 \stackrel{!}{=} 0$$

But, such $w_0^\star$ and $w_1^\star$ do not necessarily exist if $I \geq 3$!

Motion of a Projectile: *ML Approach*

How can we find w_0^* and w_1^* ? We find w_0^* and w_1^* such that

$$\sum_{i=1}^I (d_i - w_0^* v_i - w_1^* h_i)^2$$

is as small as possible

Let us define the *loss* $\mathcal{L}(w_0, w_1)$ as

$$\mathcal{L}(w_0, w_1) = \sum_{i=1}^I (d_i - w_0 v_i - w_1 h_i)^2$$

We then find w_0^* and w_1^* that minimize the *loss*

$$(w_0^*, w_1^*) = \underset{w_0, w_1}{\operatorname{argmin}} \mathcal{L}(w_0, w_1)$$

ML Components: *Dataset*

Dataset is the collection of **data-points**; in our example, the dataset was

$$\mathbb{D} = \{([v_i, h_i], d_i) : i = 1, \dots, I\}$$

which has I **data-points** $([v_i, h_i], d_i)$ for $i = 1, \dots, I$

This is an example of a **labeled dataset**: a dataset whose data-points contain both the **inputs** and their corresponding **labels (outputs)**

More general, a **labeled dataset** with I data-points is

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

- $\mathbf{x}_i \in \mathbb{R}^N$ is an input vector with N entries
- y_i is the **label** of $\mathbf{x}_i$ that is a scalar

ML Components: *Supervised Learning*

When the **dataset** is **labeled**, our *task* is clear:

- We observe I inputs to a **function** with their **outputs (labels)**
- We try to find out (**learn**) this **function**

Our task is a *learning task*, because we are trying to

*learn the unknown relation between **inputs** and **labels***

It describes a *supervised learning problem*, since

*a **supervisor** has filled our dataset with **labels***

In other words, for the sample **inputs** in our hand, we know the **outputs**

ML Components: *Unsupervised Learning*

- + But can a *dataset* be *unlabeled*?
- Yes! This is the case in *unsupervised learning* in which we are to *learn features* of a *data-point* x by investigating a set of its samples

In this case, the dataset is of the form

$$\mathbb{D} = \{x_i : i = 1, \dots, I\}$$

which is *unlabeled*. Our *learning task* is further *unsupervised*

Let's make it crystal clear via an example!

Example of Unsupervised Learning: PCA

An N -dimensional **data-point** $\mathbf{x}$ is coming from a natural process, e.g., it contains the pixel values of an image taken from body. It is known that $\mathbf{x}$ is a linear combination of $Q \ll N$ **principal vectors**: any $\mathbf{x}$ is

$$\mathbf{x} = \sum_{q=1}^Q a_q \mathbf{v}_q = [\mathbf{v}_1, \dots, \mathbf{v}_Q] \begin{bmatrix} a_1 \\ \vdots \\ a_Q \end{bmatrix} = \mathbf{V} \mathbf{a}$$

with $\mathbf{a} \in \mathbb{R}^Q$ and $\mathbf{V} \in \mathbb{R}^{N \times Q}$. Nevertheless, we do **not** know the **principle vectors**, i.e., **matrix $\mathbf{V}$ is unknown to us**.

Our **learning task** is to

find out **what $\mathbf{V}$ is** by investigating **I samples of $\mathbf{x}$**

ML Components: *Model*

Now, we know what *supervised learning* and *labeled dataset* are

Let's assume we are given by the *labeled dataset*

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

and are to learn the function that relates an *input* $\mathbf{x}$ to its *label* y

For this task we need to consider a *Model*

Model is a *parameterized* function that is used to describe the relation between the *input* $\mathbf{x}$ and its *label* y

In our *dummy example*, the model was *linear*

$$\text{label} = d = w_0 v + w_1 h = [w_0, w_1] \begin{bmatrix} v \\ h \end{bmatrix} = \mathbf{w}^T \mathbf{x}$$

ML Components: *Model*

In our **dummy example**, we considered a **linear** model, but

- we could have considered an affine model

$$\text{label} = b + \mathbf{w}^T \mathbf{x}$$

- or a polynomial model

$$\text{label} = b + \mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^2 + \dots + \mathbf{w}_P^T \mathbf{x}^P$$

Notation

By $f(\mathbf{x})$ we refer to entry-wise function operation

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \rightsquigarrow \mathbf{x}^p = \begin{bmatrix} x_1^p \\ \vdots \\ x_N^p \end{bmatrix} \quad \text{or} \quad \sqrt{\mathbf{x}} = \begin{bmatrix} \sqrt{x_1} \\ \vdots \\ \sqrt{x_N} \end{bmatrix}$$

ML Components: *Model*

In our **dummy example**, we considered a **linear** model, but

- we could have considered even a wired model

$$\text{label} = \left(\mathbf{w}_1^T \mathbf{x} \right) \left(\mathbf{w}_2^T \sqrt{\mathbf{x}} \right)$$

Models are always parameterized meaning that

they contain some parameters that are to be tuned

Example: $\mathbf{w}$ in linear model or $\mathbf{w}_p$'s and b in polynomial model

Model parameters are of two types:

- **Hyperparameters**
- **Learnable parameters**

ML Components: *Model*

Hyperparameters

Parameters that are required to specify the model *explicitly*

*Best example is the **order** P in the polynomial model: we need to know P in order to write down the model explicitly*

If we know $P = 2$; then, we know that our model is

$$\text{label} = b + \mathbf{w}_1^T \mathbf{x} + \mathbf{w}_2^T \mathbf{x}^2$$

Hyperparameters are specified prior to the start of learning process

ML Components: *Model*

Learnable Parameters

Once the **hyperparameters** are set, we have a model with some **parameters** that are to be **learned**, such that the model fits our dataset

*In our polynomial example with $P = 2$: we **learn** $\mathbf{w}_1^*$, $\mathbf{w}_2^*$, and b^* such that*

$$y_i \approx b^* + \mathbf{w}_1^{*\top} \mathbf{x}_i + \mathbf{w}_2^{*\top} \mathbf{x}_i^2$$

- + *But what does this $\approx$ mean?*
- This approximation needs to be quantified!

*This is what the **loss function** does for us*

- + *How do we **learn** the **learnable parameters**?*
- We answer this after we understand the **loss function**

ML Components: Loss

Loss Function

Loss function quantifies the difference between the *output of the model* and the *true label*

Back to our polynomial example with dataset $\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$

- We set the **hyperparameter** P to $P = 2$
- We now set the **learnable parameters**

If we give the data-point $\mathbf{x}_i$ to the model as input, it returns

$$z_i = b + \mathbf{w}_1^T \mathbf{x}_i + \mathbf{w}_2^T \mathbf{x}_i^2$$

Loss determines the difference between z_i and **true label** y_i in $\mathbb{D}$

$$\mathcal{L}(z_i, y_i) = \ell_i \in \mathbb{R}$$

ML Components: *Loss*

Examples of loss function are

1. We can calculate the *squared error*

$$\mathcal{L}(z_i, y_i) = (z_i - y_i)^2$$

which intuitively determines the **energy of the difference**

2. We can calculate the *error indicator*

$$\mathcal{L}(z_i, y_i) = \mathbb{1}(z_i \neq y_i) = \begin{cases} 1 & z_i \neq y_i \\ 0 & z_i = y_i \end{cases}$$

which indicates the **occurrence of error**

ML Components: A Quick Wrap-up

Any ML problem has *three components*:

1. *Dataset* which is the collection of samples

$$\boxed{\text{supervised learning}} \quad \mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

2. *Model* that describes the relation between the *input* and *label*

$\mathbf{h}$: Hyperparameters

$$z = f_{\mathbf{h}}(\mathbf{x} | \mathbf{w})$$

$\mathbf{w}$: Learnable Parameters

3. *Loss* quantifies the difference between the *model's output* and *true label*

$$\mathcal{L}(z_i, y_i)$$

What's Next?

Now, we know the components of an ML problem!

- + *How can we solve the problem? Or, speaking in the language of ML people, how can we address the learning task?*
- The learning task is to **tune the learnable parameters**

We approximate the label of a new input $\mathbf{x}_{new}$ as $y_{new} = f_{\mathbf{h}}(\mathbf{x}_{new} | \mathbf{w})$

The process of **tuning the learnable parameters** is called **training**

- + *How do we do the **training**?*
- We see it shortly, but first we need a more serious example!

Classification

Classification is a supervised learning problem in which

labels belong to a discrete set: $y_i \in \{c_1, \dots, c_J\}$

the label y_i represents the class to which the input x_i belongs

Best example is image classification:

- The dataset contains some images and their labels

$$\mathbb{D} = \{(\text{image 1, dog}), (\text{image 2, cat}), \dots, (\text{image } I, \text{cat})\}$$

- The images are either *dog* or *cat*

We can simply convert $\mathbb{D}$ into a numerical dataset

Classification: *Image Classification*

Any image is nothing but a *collection of pixel values*



167	163	174	168	160	162	129	161	172	161	166	166
166	182	163	74	76	62	83	17	110	210	180	164
180	180	50	14	34	6	10	33	48	106	169	181
206	109	5	124	131	111	120	204	166	15	66	180
194	68	137	251	237	239	239	226	227	87	71	201
172	105	207	233	233	214	220	239	238	98	74	206
188	88	179	209	185	215	211	168	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	168	227	178	143	182	106	36	190
205	174	165	252	236	231	149	178	238	43	96	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
196	206	123	207	177	121	123	200	176	13	96	218

167	163	174	168	160	162	129	161	172	161	166	166
166	182	163	74	76	62	83	17	110	210	180	164
180	180	50	14	34	6	10	33	48	106	169	181
206	109	5	124	131	111	120	204	166	15	66	180
194	68	137	251	237	239	239	226	227	87	71	201
172	105	207	233	233	214	220	239	238	98	74	206
188	88	179	209	185	215	211	168	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	168	227	178	143	182	106	36	190
205	174	165	252	236	231	149	178	238	43	96	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
196	206	123	207	177	121	123	200	176	13	96	218

Thus, image i in $\mathcal{D}$ can be shown by a pixel vector x_i

Classification: *Image Classification*

The *labels* can further be marked by integer numbers as *classes*

In our example, we have *two classes of dog and cat*: so, we could say

label of dog images = 0 and *label of cat images* = 1

So, a collection of I images like

$$\mathbb{D} = \{(\text{image 1, dog}), (\text{image 2, cat}), \dots, (\text{image } I, \text{cat})\}$$

can be represented by the *numerical dataset*

$$\mathbb{D} = \{(\mathbf{x}_1, 0), (\mathbf{x}_2, 1), \dots, (\mathbf{x}_I, 1)\}$$

pixel vector of image 1 it's an image of a dog

Classification: *Binary Classification*

We now start with the basic case of binary classification

Binary Classification

A classification problem with only two classes, i.e., $y_i \in \{0, 1\}$

*Our example was a binary classification with **dog: 0** and **cat: 1***

Let's build the main components of this ML problem

Binary Classification: *Dataset*

Dataset is similar to what we had in our **dog** or **cat** example

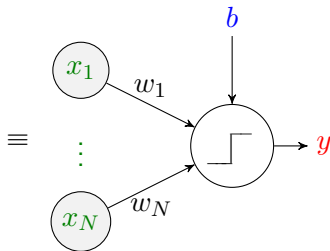
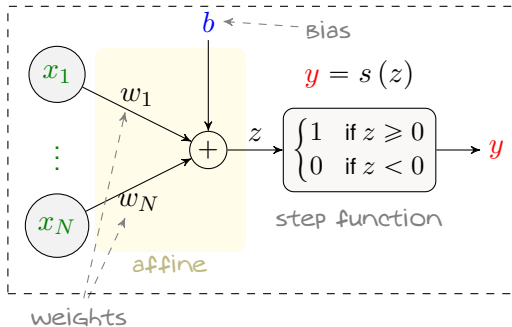
$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

- $\mathbf{x}_i \in \mathbb{R}^N$ is a real-valued N -dimensional vector
 - ▶ For instance it contains the *pixel values* of an image with N pixels
- $y_i \in \{0, 1\}$ is a **binary label**

Binary Classification: *Model*

- + What should our *model* be?
- The *model* gets N real numbers and returns a **binary number**

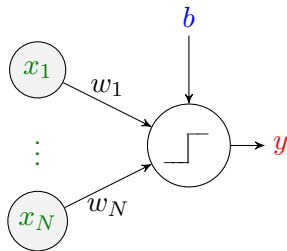
Let's start with *perceptron* who is the mother of *neural networks*



Binary Classification: *Perceptron*

Perceptron is a *linear classifier*

- it determines an *linear transform* of $\mathbf{x}$
- it *classifies* using the sign of this *transform*

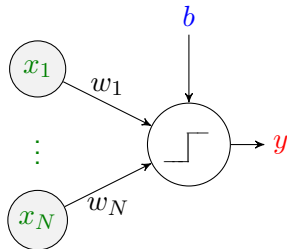


Mathematically, the perceptron is described by

$$y = s\left(\sum_{n=1}^N w_n x_n + b\right) = s\left(\underbrace{[w_1, \dots, w_N]}_{\mathbf{w}^T} \underbrace{\begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix}}_{\mathbf{x}} + b\right) = s(\mathbf{w}^T \mathbf{x} + b)$$

Binary Classification: *Perceptron*

$$y = s(\mathbf{w}^T \mathbf{x} + b)$$

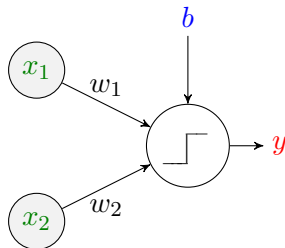


- + Is there any *hyperparameter* in this model?
- We could look at *step function* as a *hyperparameter*: we could have chosen another function to map the *affine transform* to a *binary label*
- + Is there any *learnable parameter* in this model?
- Of course! $\mathbf{w}$ and b are the *learnable parameters*

Perceptron: *Geometrical Interpretation*

Perceptron geometrically realizes a *linear division of $\mathbb{R}^N$*

To see this, let's focus on the two-dimensional case, i.e., $N = 2$



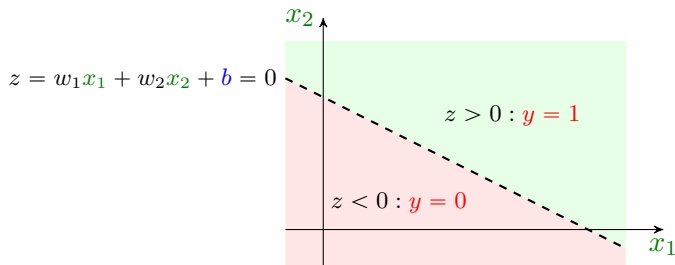
$$y = s(\mathbf{w}^T \mathbf{x} + b) = s(w_1 x_1 + w_2 x_2 + b) = \begin{cases} 1 & \text{if } w_1 x_1 + w_2 x_2 + b \geq 0 \\ 0 & \text{if } w_1 x_1 + w_2 x_2 + b < 0 \end{cases}$$

Perceptron: Geometrical Interpretation

Let's go to the two-dimensional space with one dimension denoting x_1 and the other x_2 and plot the line

$$z = w_1 x_1 + w_2 x_2 + b = 0$$

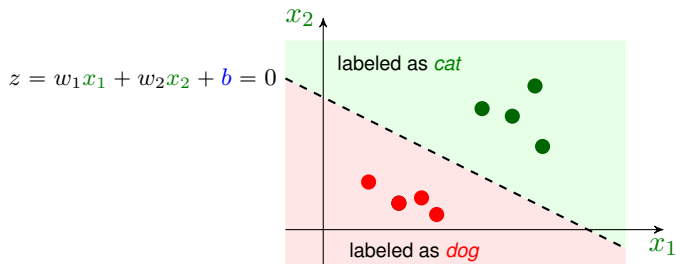
for some $w_1, w_2 > 0$ and $b < 0$



Perceptron draws a **line**, and then classifies every point above it with **label 1** and every point below it with **label 0**

Perceptron: Geometrical Interpretation

So, if we have a perceptron with *learnable parameters* $w_1, w_2 > 0$, and $b < 0$



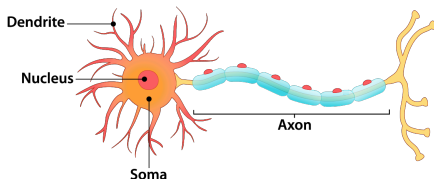
For instance in the example of image classification

- Every • represents a *data-point* $\equiv$ pixel vector: think of a two-pixel image 😊
- Perceptron gets the values of the two pixels and predicts whether it is an image of a *dog* or a *cat*

In realistic problems, this viewpoint extends to large-dimensional space

Perceptron: *A Bit of History*

- Perceptron machine was implemented by **Frank Rosenblatt** in 1957
- He wrote a paper in 1958 illustrating the machine and its algorithm
- The original idea is however older than that
 - ↳ *It was proposed in 1943 by **Warren McCulloch** and **Walter Pitts***
 - ↳ *They introduced it to abstractly describe biological neurons*
 - ↳ *This was why perceptron is also called **McCulloch-Pitts neuron***



- Perceptron was a breakthrough in the long-time ongoing attempt to understand the functionality of brain

Components of Binary Classification

Back to binary classification: we have the first two components

1. A **dataset** with N -dimensional inputs $\mathbf{x}_i$ and binary labels $y_i \in \{0, 1\}$

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

2. **Perceptron** as the **model**

$$y = s(\mathbf{w}^T \mathbf{x} + b)$$

3. For two binary variables y and $\hat{y}$, we can set the **loss** to

$$\mathcal{L}(\hat{y}, y) = \mathbb{1}(\hat{y} \neq y) = \begin{cases} 1 & \hat{y} \neq y \\ 0 & \hat{y} = y \end{cases}$$



UNIVERSITY OF
TORONTO

Training a Model

Training a Model

Up to now, we have learned the main components of ML problems and seen a classical example, i.e., classification

*We now want to understand how we can **tune** the **learnable parameters***

As mentioned, **tuning** the **learnable parameters** of a model is called
training of the model

*Before starting with **training**, let's review some basics of probability theory*

Recap: Probability Theory

Discrete random variable x is described by *probability mass function* $P(x)$

$$\Pr \{x = x_0\} = P(x_0)$$

Assuming that $x \in \mathbb{X}$, we have

$$\sum_{x \in \mathbb{X}} P(x) = 1$$

The *expectation* is also defined as

$$\mathbb{E} \{x\} = \sum_{x \in \mathbb{X}} x P(x)$$

Recap: Probability Theory

Continuous random variable x is described by *probability density function* $P(x)$

$$\Pr \{a < x \leq b\} = \int_a^b P(x) dx$$

We have in this case

$$\int_{-\infty}^{+\infty} P(x) dx = 1$$

The *expectation* is also defined as

$$\mathbb{E} \{x\} = \int x P(x) dx$$

When we talk about a *general* random variable, we call $P(x)$ the *distribution*

Recap: Probability Theory

A random vector $\mathbf{x}$ is a vector of random variables and its distribution

$$P(\mathbf{x}) = P(x_1, \dots, x_N)$$

is the *joint distribution of the entries*

Also, we can take expectation of any function of $\mathbf{x}$ as

$$\mathbb{E} \{f(\mathbf{x})\} = \int f(\mathbf{x}) P(\mathbf{x}) d\mathbf{x}$$

Recap: Law of Large Numbers

Consider a random sequence $x_1, \dots, x_I$

we call it *independent* and *identically distributed* (i.i.d.) with $x \sim P(x)$

if x_i 's are generated *independently* all with the *same distribution* $P(x)$

Law of Large Numbers

Assume $x_1, \dots, x_I$ is an i.i.d. sequence with $x \sim P(x)$; then, we have¹

$$\frac{1}{I} \sum x_i \rightarrow \mathbb{E} \{x\}$$

as $I \rightarrow \infty$

Intuition. If we *arithmetically average* too many samples of a random process, we get a value very close to its *expectation*

¹Of course under some conditions which we assume holding

Training a Model

Let us now use the probability theory to derive a meaningful approach of *model training*: for sake of simplicity, let's assume that we have *dataset*

$$\mathbb{D} = \{(x_i, y_i) : i = 1, \dots, I\}$$

with *scalar (one-dimensional) inputs*, i.e., x_i 's are real numbers

Let us further denote our model as

$$y = f(x|\mathbf{w})$$

where in this notation

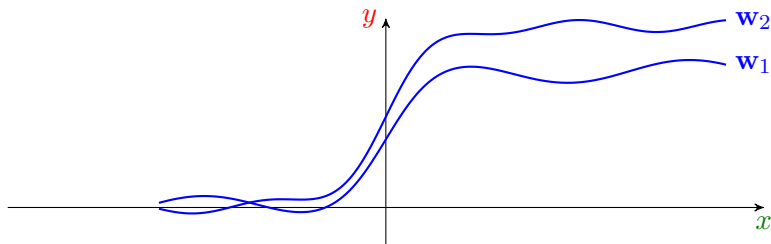
- $\mathbf{w}$ contains all *learnable* parameters that are to be learned in *training*
- we drop the *hyperparameters*, since *they are yet fixed by genie*

Training a Model

The *one-dimensional* assumption helps us visualize the model

$$y = f(x|\mathbf{w})$$

With scalar *input*, we can visualize the model as



As *learnable* parameters change, the model represents different functions

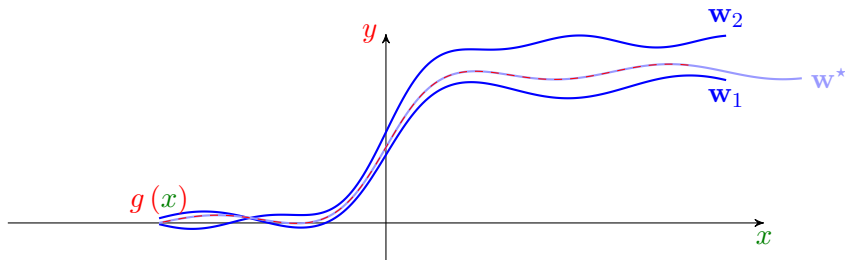
Training a Model

In reality, the *labels* and *inputs* are related through a function $g(\cdot)$

$$y = g(x) \rightsquigarrow \text{function } g(\cdot) \text{ is however } \textit{unknown} \text{ to us}$$

In fact, *our whole learning task* is to *learn* it from the *dataset*!

- + What if we knew *function* $g(\cdot)$? How would have we trained our model?
- Well! We would have tuned w until the model matches $g(\cdot)$



Training a Model

We could represent such $\mathbf{w}^*$ using the *loss*

Point-wise Loss

The model with *parameter* $\mathbf{w}$ gives us the label $\hat{y}_0 = f(x_0 | \mathbf{w})$ for *input* x_0

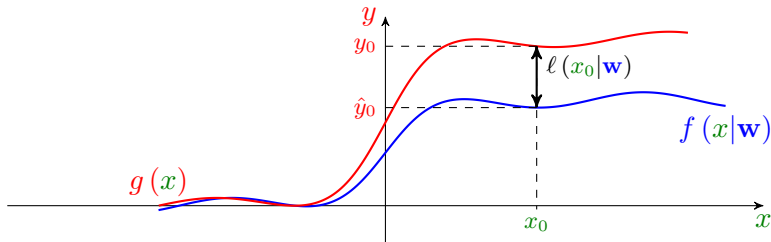
- It may be different from the *true label* $y_0 = g(x_0)$
- The *loss* between the two is given by

$$\ell(x_0 | \mathbf{w}) = \mathcal{L}(\hat{y}_0, y_0) = \mathcal{L}(f(x_0 | \mathbf{w}), g(x_0))$$

Training a Model: *Point-wise Loss*

$$\ell(x_0|\mathbf{w}) = \mathcal{L}(\hat{y}_0, y_0) = \mathcal{L}(f(x_0|\mathbf{w}), g(x_0))$$

Let's visualize this loss



Training a Model: *Risk*

We would like our *model* to recover the *true label* as closely as possible: so, the best option at point x_0 is to find the choice of $\mathbf{w}$ that minimizes $\ell(x_0|\mathbf{w})$

- + But x_0 is a *single* point! What about other *inputs*?!
- Right! We should *learn* $\mathbf{w}$ for any x_0
- + How can we do it?
- We treat x_0 as a random variable with some distribution $P(x_0)$, and minimize the *expexted loss* often called *risk*

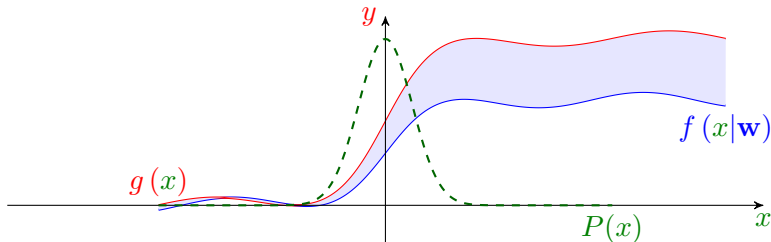
Risk

For given learnable parameter $\mathbf{w}$, the *risk* is defined as

$$R(\mathbf{w}) = \mathbb{E} \{ \ell(x_0|\mathbf{w}) \} = \int \ell(x_0|\mathbf{w}) P(x_0) dx_0$$

Training a Model: *Risk Minimization*

Let's visualize the *risk*



Risk Minimization

Ideally, the training is formulated as the minimization of *risk*

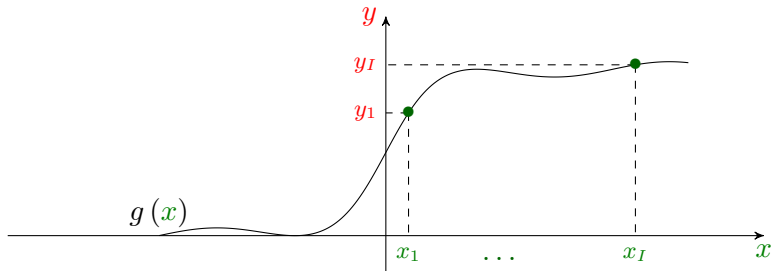
$$\mathbf{w}^{\star} = \underset{\mathbf{w}}{\operatorname{argmin}} R(\mathbf{w})$$

Training a Model: *Empirical Risk*

- + Bravo! But, the *training* seems *infeasible*, since we have neither the *true function* $g(\cdot)$, nor the *distribution* $P(x)$!
- Right! But, we could handle this approximately using the *LLN*

Let's look at what we have: the *dataset* which contains *samples* of $g(\cdot)$

$$\mathbb{D} = \{(x_i, y_i) : i = 1, \dots, I\}$$



Training a Model: *Empirical Risk*

We can determine *point-wise losses* for points in the *dataset*

$$\ell(x_i|\mathbf{w}) = \mathcal{L}(f(x_i|\mathbf{w}), g(x_i)) = \mathcal{L}(f(x_i|\mathbf{w}), y_i)$$

Now, assume that data-points are drawn from i.i.d. *unknown* $P(x)$

LLN suggests that *average* of *point-wise losses* converges to the *risk* when we have a large enough number of data-points, i.e.

$$\frac{1}{I} \sum_{i=1}^I \ell(x_i|\mathbf{w}) \rightarrow \mathbb{E} \{ \ell(x|\mathbf{w}) \} = R(\mathbf{w})$$

We call this *arithmetic average* the *empirical risk*

Empirical risk is the best *estimate* of *risk* that we get from our *dataset*

Training a Model: *Empirical Risk Minimization*

Empirical Risk

Let $\mathbf{w}$ includes all learnable parameters of the model, and the dataset be

$$\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$$

for loss function $\mathcal{L}$, the empirical risk is defined as

$$\hat{R}(\mathbf{w}) = \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f(\mathbf{x}_i | \mathbf{w}), y_i)$$

Model Training: *Empirical Risk Minimization*

The *training* is performed by minimizing the empirical risk

Empirical Risk Minimization

Training model on a *dataset* $\equiv$ *minimizing empirical risk* computed from the *dataset*

$$\mathbf{w}^{\star} = \underset{\mathbf{w}}{\operatorname{argmin}} \hat{R}(\mathbf{w}) \quad (\text{Training})$$

$$= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathcal{L}(f(\mathbf{x}_i | \mathbf{w}), y_i)$$

Wrap-Up: *Universal Scheme for ML*

So, we know pretty much all the theory for *supervised* learning:

1. We build the main three components:
 - 1.1 Dataset
 - 1.2 Model
 - 1.3 Loss Function
2. We determine the *empirical risk* of the *model* using *loss function*
3. We *train* the *model* by minimizing the *empirical risk*

We call this universal approach ML 1-2-3!

The theory is pretty short; however,

how to execute each step of ML 1-2-3 is a pretty long story

that we are going to learn in the remaining of this course!



UNIVERSITY OF
TORONTO

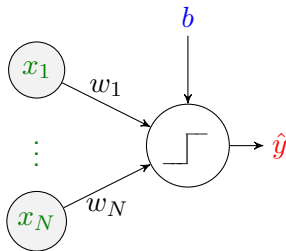
Universal Approximation

Training Perceptron Machine: *ML 1*

Let's start our long story with *binary classification* via *perceptron*

1. Dataset $\mathbb{D} = \{(\mathbf{x}_i, y_i) : i = 1, \dots, I\}$ with *binary labels* $y_i \in \{0, 1\}$
2. *Perceptron* as the *model*

$$\hat{y} = s(\mathbf{w}^T \mathbf{x} + b) = \begin{cases} 1 & \text{if } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{if } \mathbf{w}^T \mathbf{x} + b < 0 \end{cases}$$



3. *Error indicator* as the *loss* function

$$\mathcal{L}(\hat{y}, y) = \mathbb{1}(\hat{y} \neq y) = \begin{cases} 1 & \hat{y} \neq y \\ 0 & \hat{y} = y \end{cases}$$

Training Perceptron Machine: *ML 2*

Let's write down the *empirical risk*

$$\begin{aligned}\hat{R}(\mathbf{w}, b) &= \frac{1}{I} \sum_{i=1}^I \mathcal{L}(\hat{y}_i, y_i) \\ &= \frac{1}{I} \sum_{i=1}^I \mathbb{1}(\hat{y}_i \neq y_i) \\ &= \frac{1}{I} \sum_{i=1}^I \mathbb{1}(s(\mathbf{w}^\top \mathbf{x}_i + b) \neq y_i) \\ &= \frac{\text{\# of times perceptron misclassifies}}{I} \\ &\equiv \text{\textit{Error Rate}}\end{aligned}$$

Training Perceptron Machine: *ML 3*

We should now minimize the *empirical risk*

$$\mathbf{w}^*, b^* = \underset{\mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}}{\operatorname{argmin}} \hat{R}(\mathbf{w}, b) = \underset{\mathbf{w} \in \mathbb{R}^N, b \in \mathbb{R}}{\operatorname{argmin}} \frac{1}{I} \sum_{i=1}^I \mathbb{1}(s(\mathbf{w}^\top \mathbf{x}_i + b) \neq y_i)$$

But, how can we solve this optimization? It *doesn't* look *easy*!

Let's see what Rosenblatt did: *Rosenblatt's perceptron had no bias, i.e., $b = 0$*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^\top \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \operatorname{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

Training Perceptron Machine: *Perceptron Algorithm*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^\top \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

- Outer loop stops only if $\hat{R}(\mathbf{w}) = 0$ which is minimum empirical risk
- In inner loop, let's say at iteration t error occurs for $\mathbf{x}_i$

↳ **either** $z_i = \mathbf{w}_t^\top \mathbf{x}_i > 0$ and $y_i = 0$: we update as $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \mathbf{x}_i$

$$\mathbf{w}_{t+1}^\top \mathbf{x}_i = (\mathbf{w}_t - \mathbf{x}_i)^\top \mathbf{x}_i = \mathbf{w}_t^\top \mathbf{x}_i - \|\mathbf{x}_i\|^2 \leq \mathbf{w}_t^\top \mathbf{x}_i$$

so the algorithm pushes the weights somewhere that z_i could get *negative*

Training Perceptron Machine: *Perceptron Algorithm*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^\top \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

- Outer loop stops only if $\hat{R}(\mathbf{w}) = 0$ which is minimum empirical risk
- In inner loop, let's say at iteration t error occurs for $\mathbf{x}_i$

↳ **or** $z_i = \mathbf{w}_t^\top \mathbf{x}_i < 0$ and $y_i = 1$: we update as $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \mathbf{x}_i$

$$\mathbf{w}_{t+1}^\top \mathbf{x}_i = (\mathbf{w}_t + \mathbf{x}_i)^\top \mathbf{x}_i = \mathbf{w}_t^\top \mathbf{x}_i + \|\mathbf{x}_i\|^2 \geq \mathbf{w}_t^\top \mathbf{x}_i$$

so the algorithm pushes the weights somewhere that z_i could get *positive*

Training Perceptron Machine: *Perceptron Algorithm*

```
1: Start with  $\mathbf{w} = \mathbf{0}$  or some small random initial value
2: while  $\hat{R}(\mathbf{w}) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^T \mathbf{x}_i$  and  $\hat{y}_i = s(z_i)$       # pass through perceptron
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$ 
7:     end if
8:   end for
9: end while
```

- + *It makes sense that in each iteration $\mathbf{w}$ gets modified towards a **right direction**! But can we guarantee that this algorithm always converges?*
- Well! Let's try some examples

Perceptron Algorithm: *AND Operator*

Assume that we have the following **dataset** two-dimensional **inputs**

$$\mathbb{D} = \left\{ \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, 0 \right), \left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, 0 \right), \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}, 0 \right), \left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1 \right) \right\}$$

We intend to **train perceptron** with this **dataset** via **perceptron algorithm**

*Before we start **training**, we note that this dataset represents the **AND operator***

$$\forall (\mathbf{x}_i, y_i) \in \mathbb{D} : y_i = x_{i,1} \wedge x_{i,2}$$

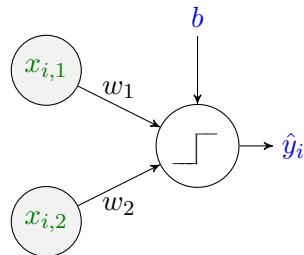
*so, we basically want to see, if we could realize this operator via **perceptron***

Perceptron Algorithm: *AND* Operator

```

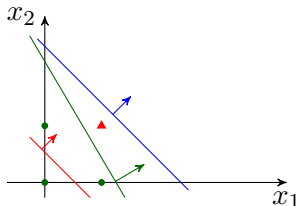
1: Start with  $\mathbf{w} = [1.1, 1.1]^T$  and  $b = -2.25$ : blue
2: while  $\hat{R}(\mathbf{w}, b) \neq 0$  do
3:   for  $i = 1 : I$  do
4:     Compute  $z_i = \mathbf{w}^T \mathbf{x}_i + b$  and  $\hat{y}_i = s(z_i)$ 
5:     if  $\hat{y}_i \neq y_i$  then
6:        $\mathbf{w} \leftarrow \mathbf{w} - \text{sgn}(z_i) \mathbf{x}_i$  and  $b \leftarrow b - \text{sgn}(z_i)$ 
7:     end if
8:   end for
9: end while

```



Let's show data-points with $y_i = 1$ by $\blacktriangle$ and those with $y_i = 0$ by $\bullet$

- Updated by $\mathbf{x}_i = [1, 1]^T$
 $\hookrightarrow \mathbf{w} = [2.1, 2.1]^T$ and $b = -1.25$: red
- Updated by $\mathbf{x}_i = [0, 1]^T$
 $\hookrightarrow \mathbf{w} = [2.1, 1.1]^T$ and $b = -2.25$: green



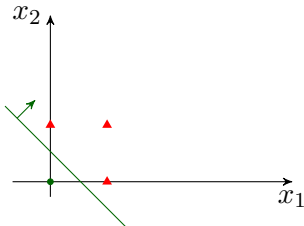
Perceptron Algorithm: *AND* Operator

Great! We *trained* the *perceptron* to behave as *logical AND*

- + What about *logical OR*?
- Easy! Let's write the *dataset*

$$\mathbb{D} = \left\{ \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, 0 \right), \left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, 1 \right), \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}, 1 \right), \left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}, 1 \right) \right\}$$

Trying the *perceptron algorithm*, we end up with some *linear classifier* like

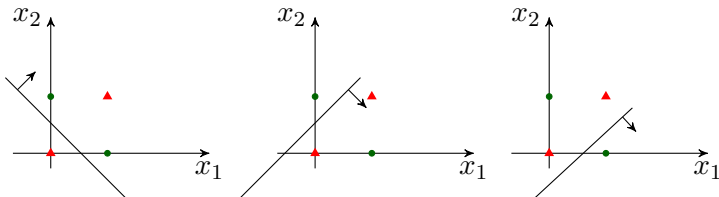


Perceptron Algorithm: *False Conclusion*

A false initial conclusion is that *perceptron* can realize *any* binary function! But, it is easy to see that *this is not the case!* Let's consider **logical XOR**:

$$x_1 \oplus x_2 = \begin{cases} 1 & \text{if } x_1 \neq x_2 \\ 0 & \text{if } x_1 = x_2 \end{cases} \rightsquigarrow \mathbb{D} = \left\{ \left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, 0 \right), \left(\begin{bmatrix} 0 \\ 1 \end{bmatrix}, 1 \right), \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}, 1 \right), \left(\begin{bmatrix} 1 \\ 1 \end{bmatrix}, 0 \right) \right\}$$

It's not hard to see that *perceptron* cannot learn this function²



²If you don't see it clearly, no worries! You'll show it as an assignment

Multi-Layering Perceptrons

Let's play a bit with *logical XOR*: recall that

$$x_1 \oplus x_2 = \begin{cases} 1 & \text{if } x_1 \neq x_2 \\ 0 & \text{if } x_1 = x_2 \end{cases}$$

We can write this function as

$$x_1 \oplus x_2 = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$$

where we have used the following notation

$$\bar{x} \equiv \text{complement of } x = 1 - x \quad \text{and} \quad \vee \equiv \text{logical OR}$$

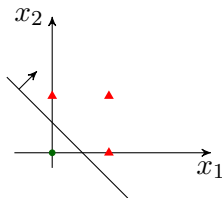
Multi-Layering Perceptrons

Logical XOR expands as

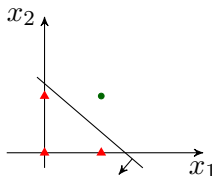
$$y = x_1 \oplus x_2 = \underbrace{(x_1 \vee x_2)}_{h_1} \wedge \underbrace{(\bar{x}_1 \vee \bar{x}_2)}_{h_2} = h_1 \wedge h_2$$

We can learn h_1 and h_2 by two different *perceptrons*

$$h_1 = x_1 \vee x_2$$

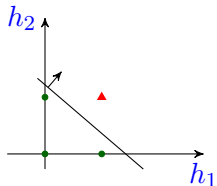


$$h_2 = \bar{x}_1 \vee \bar{x}_2 \equiv \overline{x_1 \wedge x_2}$$

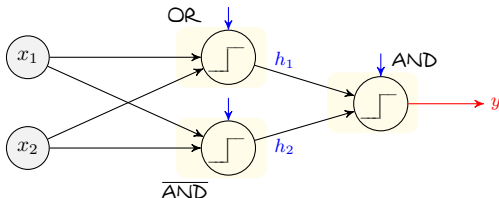


Multi-Layering Perceptrons

We can further learn $y = h_1 \wedge h_2$ by another *perceptron*

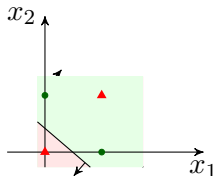


Well! It's true that we cannot learn **XOR** by a single *perceptron*, but we can learn it with a *network of three perceptrons*!



Multi-Layering Perceptrons: Geometrical Interpretation

Let's see geometrically what this network of *perceptrons* does



- First *perceptron* classifies
- Second *perceptron* classifies
- Third *perceptron* intersects the two regions

Multi-Layering Perceptrons: *Correct Conclusion*

We can learn *any* binary function using a *network of perceptrons*!

This has given birth to the idea of *neural network*

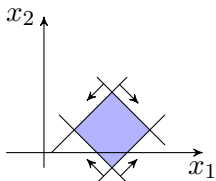
Neural Networks: *Intuitive Introduction*

looking at *perceptron* as an *artificial neuron*, we can *learn* any complicated function when we have a network of these *neurons* $\equiv$ a *Neural Network*

- + *But, why should we care only about binary functions? Don't we learn other type of functions as well?*
- Well! We can extend the idea to other problems as well!

Multi-Layering Perceptrons: *More Complicated Functions*

Assume we have *binary classification* with *real-valued inputs*: we want to train a classifier that distinguishes between the *two-dimensional points* inside the *blue area* and outside of it



- We classify *Region 1* with three *perceptrons*
- We classify *Region 2* with three *perceptrons*
- We *intersect* the two *regions* with a *perceptron*

*A network of 7 *perceptrons* is more than enough!*

Multi-Layering Perceptrons: *More Complicated Functions*

- + What if the classification problem is *not binary*?
- Well! We already have seen that we can reduce a **multi-class** classification to a series of binary classifications³

We want to classify an image as *dog, cat or car*

- ▶ *Binary Classification 1: Is it class 0: dog or class 1: {cat, car}?*
 - ↳ If *class 0* $\rightsquigarrow$ classification ended
 - ↳ If *class 1* $\rightsquigarrow$ *Binary Classification 2: Is it class 0: cat or class 1: car?*

Moral of Story

We can learn *any classifier* with *high accuracy* using a network of *perceptrons*

³There are better **multi-class** classification techniques that we learn later

Multi-Layering Perceptrons: *More Complicated Functions*

- + *Fair enough! We are happy with classification! But, what about the case that we have **real-valued labels**? Can we do it by **perceptrons**?*
- This is the **regression problem**!
- And, Yes! We can do this as well using **perceptrons**

Regression is a **supervised** learning problem in which

***labels** belong to a continuous set, e.g., $y_i \in \mathbb{R}$*

the **label** y_i in this case is the function sample at x_i

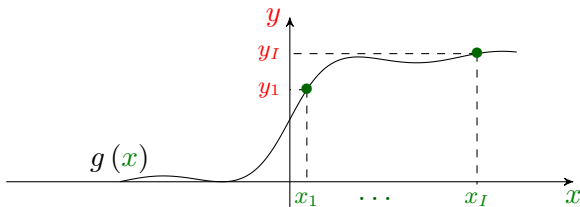
Multi-Layering Perceptrons: *Regression*

In *regression*, we learn a *real-valued function*

Let's look at a simple case with *one-dimensional inputs*, where we can visualize

$$\mathbb{D} = \{(x_i, y_i) : i = 1, \dots, I\}$$

A visualization of this *dataset* is



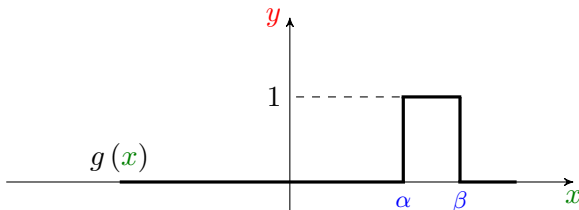
and we want to *learn* $g(\cdot)$

Multi-Layering Perceptrons: *Regression*

The main question is

can we realize an *arbitrary* $g(\cdot)$ via a network of *perceptrons*?

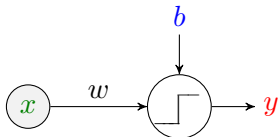
Let's start with a simple function: *the unit pulse*



We can realize this function using three *perceptrons*

Multi-Layering Perceptrons: *Regression*

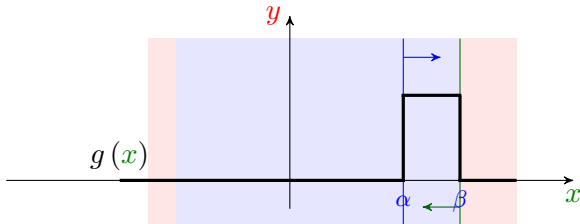
First, let's recall how **perceptron** looks with **one-dimensional** input



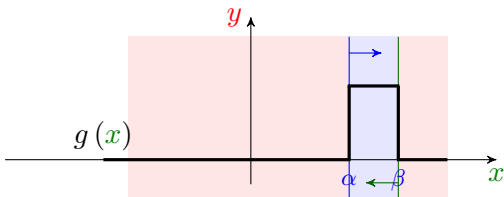
↳ with $w = 1$ and $b = -\alpha$: **blue**

↳ with $w = -1$ and $b = \beta$: **green**

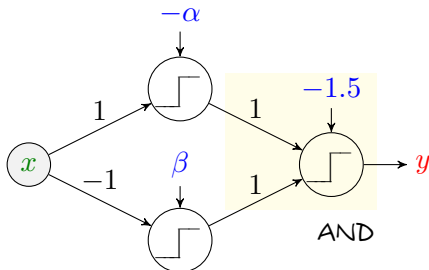
We realize the pulse by applying **AND** on the outputs of these **perceptrons**



Multi-Layering Perceptrons: *Regression*

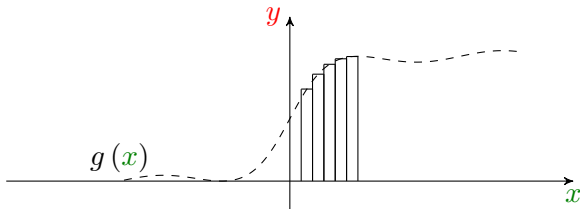


So, we can realize the unit pulse via the following *network of perceptrons*



Multi-Layering Perceptrons: *Regression*

We can approximate a *general* function via a *weighted sum* of unit pulses



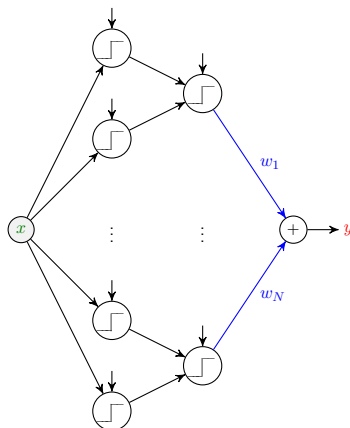
This means that we can approximate *any* function by

- realizing each pulse by a *network of perceptrons*
- applying *linear transform* on the *outputs* of these *networks*

Multi-Layering Perceptrons: *Regression*

Real-valued functions are *well-approximated* by

a network of *perceptrons* + a *linear transform*



Multi-Layering Perceptrons: *Summary*

A network of *perceptrons* seems to be a very sophisticated *model*

- It can *learn any classifier*
- It can *approximate any function* with *arbitrary accuracy*

Universal Approximation Theorem (*informal*)

Given function $g(\cdot)$ and $\varepsilon > 0$, there exists a *neural network* $f_{\mathbf{h}}(\cdot|\mathbf{w})$ that

$$\sup_{\mathbf{x}} \|g(\mathbf{x}) - f_{\mathbf{h}}(\mathbf{x}|\mathbf{w})\| \leq \varepsilon$$

So, it seems natural to train them for our learning tasks

*A network of **perceptrons** is a special **artificial neural network***
*we now formally introduce **artificial neural network***



UNIVERSITY OF
TORONTO

Further Read

Further Read

Read More on These Topics



Goodfellow, et al., *Deep Learning*

↳ [Chapters 2 and 3](#)



Bishop & Bishop, *Deep Learning: Foundations and Concepts*

↳ [Chapters 2 and 5](#)

Nice to Check

1. *A logical calculus of the ideas immanent in nervous activity*
↳ McCulloch and Pitts (1943)
2. *An overview of statistical learning theory*
↳ Vapnik (1999)